

מבוא למדעי המחשב

שיעור 9: חריגות

```
def get_int():  
    result = int(input("please enter a number:"))  
    return result
```

• ומה אם המשתמש מקליד אות במקום מספר?

טיפול בחריגה

```
def get_int():  
    while True:  
        try:  
            result = int(input("please enter a number: "))  
            return result  
        except:  
            print("oops! you did not mean that.")  
  
x = get_int()  
print("got", x)
```

רצוי לציין חריגה ספציפית

```
def get_int():  
    while True:  
        try:  
            result = int(input("please enter a number: "))  
            return result  
        except ValueError:   
            print("oops! you did not mean that.")  
  
x = get_int()  
print("got", x)
```

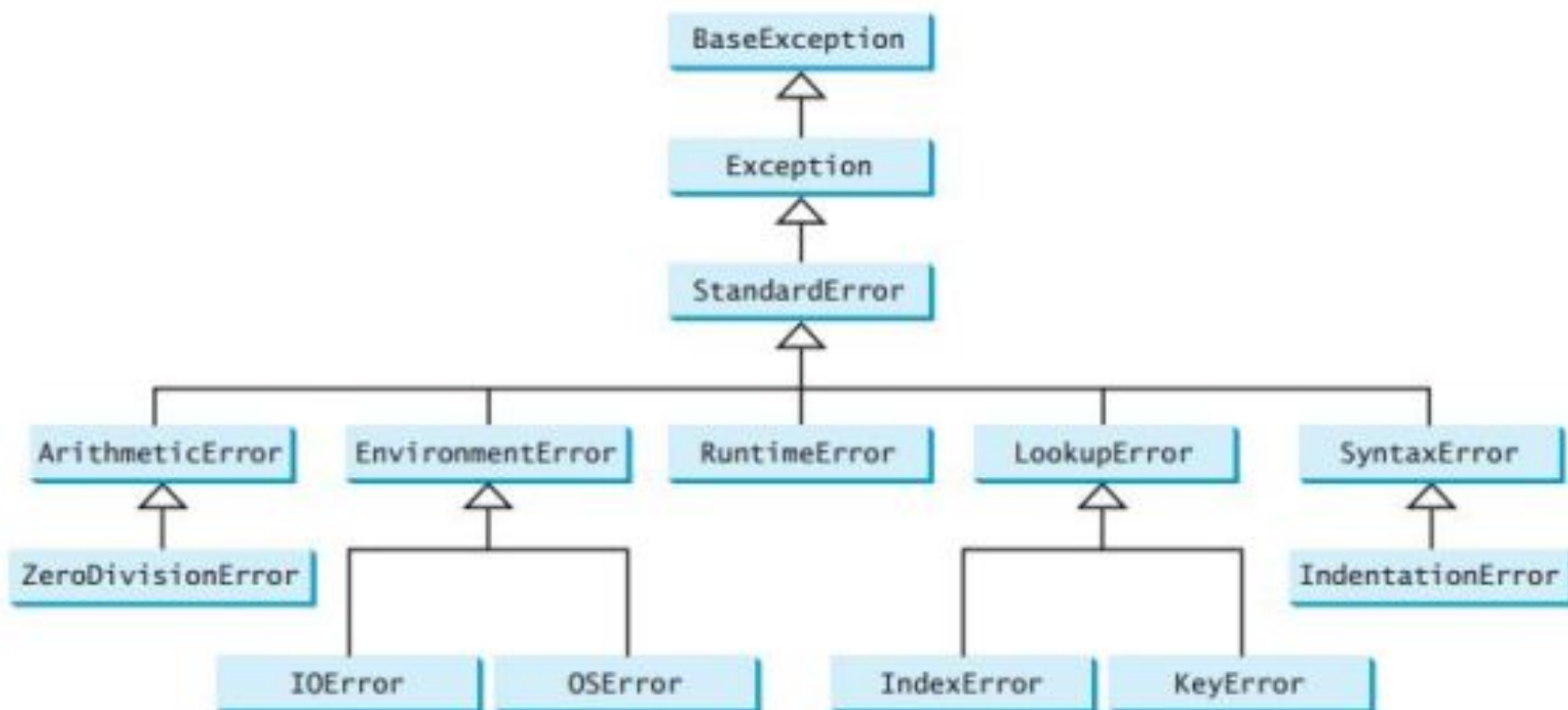
```
please enter a number: a  
oops! you did not mean that.  
please enter a number: x  
oops! you did not mean that.  
please enter a number: b  
oops! you did not mean that.  
please enter a number: 1  
got 1
```

יש הרבה טיפוסים חריגות

• ראו כאן:

<https://docs.python.org/3/library/exceptions.html>

היררכיה של חריגות



טיפול במספר חריגות

```
def multi_exception_example(lst, index):  
    try:  
        return 1/int(lst[index])  
    except ValueError:  
        print("Value error")  
    except (IndexError, ZeroDivisionError):  
        print("Index error or zero division")
```

```
lst = ["a", "1", "0"]  
multi_exception_example(lst, 0)  
multi_exception_example(lst, 1)  
multi_exception_example(lst, 2)  
multi_exception_example(lst, 3)
```

```
def multi_exception_example(lst, index):  
    try:  
        return 1/int(lst[index])  
    except ValueError as err:  
        print("Value error:\n  ", err)  
    except (IndexError, ZeroDivisionError) as err:  
        print("Index error or zero division:\n  ", err)
```

```
lst = ["a", "1", "0"]  
multi_exception_example(lst, 0)  
multi_exception_example(lst, 1)  
multi_exception_example(lst, 2)  
multi_exception_example(lst, 3)
```

```
Value error:  
    invalid literal for int() with base 10: 'a'  
Index error or zero division:  
    division by zero  
Index error or zero division:  
    list index out of range
```

העלאת חריגה

```
def calculate(num1, op, num2):  
    if op == "*":  
        return num1*num2  
    if op == "+":  
        return num1+num2  
    if op == "-":  
        return num1-num2  
    if op == "/":  
        return num1/num2  
    else:  
        raise Exception("Unknown operation", op)
```

- רצוי לפרט חריגה יותר ספציפית (למה?)

חריגות מתגלגלות במורד מחסנית זמן הריצה

```
def func1():  
    func2()
```

```
def func2():  
    raise Exception
```

```
try:  
    func1()  
except Exception:  
    print("caught outside")
```

עוד דוגמה

```
def func1():  
    try:  
        func2()  
    except ValueError:  
        print("a value error")
```

```
def func2():  
    raise Exception  
  
try:  
    func1()  
except Exception:  
    print("caught outside")
```

עקרונות הטיפול בחריגות

- מתוך הזן של פייתון:

```
Errors should never pass silently.  
Unless explicitly silenced.
```

- מעלים חריגה כאשר אי אפשר לטפל בבעייה בפונקציה שקרתה בה (אבל הפונקציה הקוראת יכולה)

- מטפלים בחריגה מוקדם ככל האפשר
– משתדלים שהחריגה לא תתגלגל ל-main

- מנקים את משתני המצב תוך כדי הטיפול בחריגה
– שומרים על קונסיסטנטיות האובייקטים
– ייתכן שהתוכנית תמשיך בריצה!

```
def enter_age(age):  
    if age < 0:  
        raise ValueError("Only positive integers are allowed")  
    if age % 2 == 0:  
        print("age is even")  
    else:  
        print("age is odd")
```

```
raise ValueError  
raise ValueError()
```

אלו שקולים

- חריגות הן מנגנון מעולה להשתמש בזמן יצירת אובייקט, אם מקבלים פרמטרים שגויים:

```
class FIFOBuffer:
    def __init__(self, buffer_size=DEFAULT_BUFFER_SIZE):
        if buffer_size < 0:
            raise ValueError("Buffer size cannot be negative")

        self.__buffer = [None]*buffer_size
```

הגדרת מחלקות חדשות של חריגות

דורש את מנגנון הירושה (לא בחומר קורס זה)
בערך (אבל לא בדיוק) כך:

```
class InputError(Exception):  
    """Exception raised for errors in the input.  
  
    Attributes:  
        expression -- input expression in which the error occurred  
        message -- explanation of the error  
    """  
  
    def __init__(self, expression, message):  
        self.expression = expression  
        self.message = message
```

finally-1 else

```
def get_int():  
    try:  
        #some thing to do  
        result = int(input("Enter a number: "))  
    except ValueError:  
        # if there is an error:  
        print("oops!")  
    else:  
        # if there was no exception  
        result = result*2  
        return result  
    finally:  
        # do this ALWAYS  
        print("hello")
```

```
x = get_int()  
print("got", x)
```

```
Enter a number: a  
oops!  
hello  
got None
```

```
Enter a number: 2  
hello  
got 4
```

```
def get_int():  
    try:  
        #some thing to do  
        result = int(input("Enter a number: "))  
    finally:  
        # do this ALWAYS  
        print("must clean here!")
```

Enter a number: *a*

must clean here!

Traceback (most recent call last):

File "[C:/Users/avivz/Desktop/tmp/exceptions/except.py](#)", line 9, in <module>
 x = get_int()

File "[C:/Users/avivz/Desktop/tmp/exceptions/except.py](#)", line 4, in get_int
 result = int(input("Enter a number: "))

ValueError: invalid literal for int() with base 10: 'a'

העלאה מחדש של חריגה

```
try:
    f = open('integers.txt')
    s = f.readline()
    i = int(s.strip())
except IOError as e:
    print(e)
except ValueError:
    print("No valid integer in line.")
except:
    print("Unexpected error opening or reading file.")
    raise 
```

החריגה ממשיכה להתגלגל במורד המחסנית

אל תעשו זאת!

```
def sum_list(lst):  
    index = 0  
    result = 0  
    try:  
        while True:  
            result += lst[index]  
            index += 1  
    except IndexError:  
        return result
```

- אין להשתמש במנגנון החריגות כחלק מהמהלך התקין של הקוד, אלא רק לטיפול בשגיאות

איטורים

• תזכורת:

```
In[2]: names = ["Alice", "Bob", "Charlie"]
```

```
In[3]: x = iter(names)
```

```
In[4]: next(x)
```

```
Out[4]: 'Alice'
```

```
In[5]: next(x)
```

```
Out[5]: 'Bob'
```

```
In[6]: next(x)
```

```
Out[6]: 'Charlie'
```

```
In[7]: next(x)
```

```
Traceback (most recent call last):
```

```
File "C:\WinPython-64bit-3.3.5.0\python-3.3.5.amd64\lib\site-
```

```
exec(code_obj, self.user_global_ns, self.user_ns)
```

```
File "<ipython-input-7-5e4e57af3a97>", line 1, in <module>
```

```
next(x)
```

```
StopIteration
```

iterable , iterator

```
class my_class:
```

```
    def __iter__(self):  
        ....
```

מחזיר איטרטור

הופך את המחלקה
ל-iterable

```
class my_class_iter:
```

```
    def  
    __next__(self):  
        ....
```

```
    def __iter__(self):  
        return self
```

גם איטרטורים הם
iterable

המתודה מחזירה
את האיבר הבא

איטרטור מספרים ראשוניים

```
class PrimesIter():  
    def __init__(self, max_num):...  
  
    def __iter__(self):...  
  
    def __next__(self):...  
  
def is_prime(num):...
```

```
def is_prime(num):  
    for i in range(2, int(num**(1/2)) + 1):  
        if num%i==0:  
            return False  
    return True
```

```
class PrimesIter():  
    def __init__(self, max_num):  
        self.__max = max_num  
        self.__cur = 1  
  
    def __iter__(self):  
        return self
```

```
def __next__(self):  
    self.__cur += 1  
    if self.__cur >= self.__max:  
        raise StopIteration  
    while not is_prime(self.__cur):  
        self.__cur += 1  
        if self.__cur >= self.__max:  
            raise StopIteration  
    return self.__cur
```

זו הדרך להכריז שהגענו לסוף הסדרה